Advanced Methods in Deep Learning

Lecture 1: Introduction to Deep Learning and Perceptrons

Alexander Quispe Rojas
Universidad del Pacífico – MECA
aw.quispero@up.edu.pe aquisper@caltech.edu

April 13, 2026

Today's Roadmap

Course Overview and Motivation

The Supervised Learning Problem

Linear Models as Starting Point

The Perceptron

From Perceptrons to Deep Networks

Introduction to PyTorch and Google Colab

Summary and Next Steps

Course Overview & Motivation

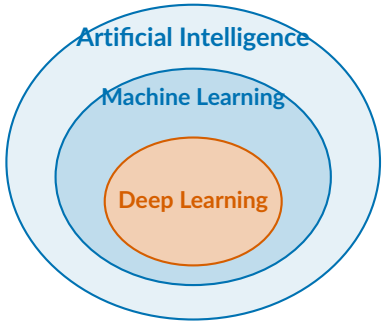
What is Deep Learning?

Machine Learning: algorithms that learn from data.

Deep Learning: a subset of ML that uses neural networks with multiple layers to learn hierarchical representations of data.

Key idea:

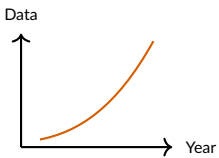
- Instead of hand-engineering features, the model *learns* the right representation.
- Each layer transforms the data into a more useful representation.
- **Depth** = number of stages of processing.



Why Deep Learning Now?

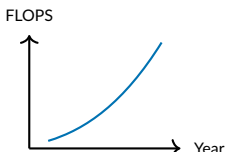
Three ingredients came together:

1. Data



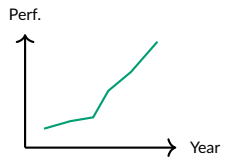
Internet, digitization, sensors, satellites

2. Compute (GPUs)



NVIDIA GPUs, Google TPUs, cloud computing

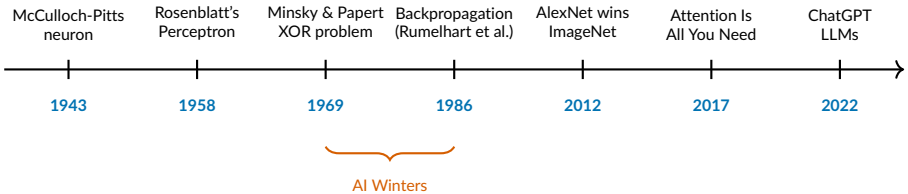
3. Algorithms



Backpropagation, ReLU, dropout, Transformers

Neural networks existed since the 1950s — what changed is the scale.

A Brief History of Neural Networks



- **1943–1969:** Early excitement $\rightarrow$ crushed by XOR limitation.
- **1986:** Backpropagation revives interest, but limited by compute.
- **2012:** Deep CNNs dominate computer vision (AlexNet).
- **2017–now:** Transformers $\rightarrow$ LLMs $\rightarrow$ foundation models revolution.

Deep Learning in Economics – Why Should You Care?

Text as Data

- Sentiment analysis of central bank minutes.
- Classification of policy documents.
- NLP on congressional speeches.

Hansen & McMahon (2016), Gentzkow et al. (2019)

Prediction & Forecasting

- GDP nowcasting with unstructured data.
- Financial market prediction.
- Consumer behavior modeling.

Athey & Imbens (2019)

Satellite Imagery

- Predicting poverty from nightlight data.
- Measuring economic activity.
- Agricultural output estimation.

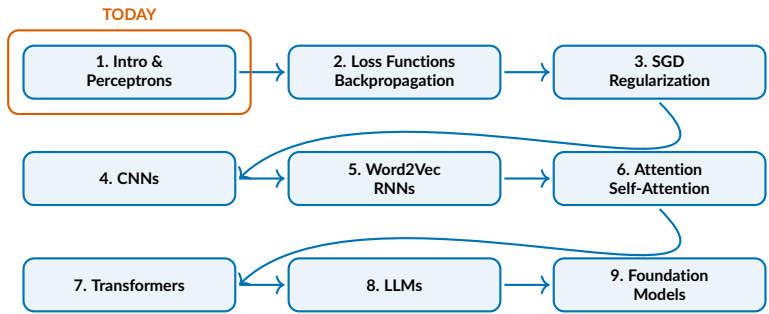
Jean et al. (2016, *Science*)

Causal Inference + DL

- Double/debiased ML with neural nets.
- Heterogeneous treatment effects.
- Instrumental variables with DL.

Chernozhukov et al. (2018)

Course Roadmap



Evaluation: 4 Homeworks (60%) + Oral Exam (30%) + Participation (10%)
Tools: Python, PyTorch, Google Colab — LLMs encouraged (Claude Code, ChatGPT)

The Supervised Learning Problem

Supervised Learning: The Setup

The Learning Problem

Given:

- A dataset of N input-output pairs: $\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)\}$.
- Each input: $\mathbf{x}_i \in \mathbb{R}^d$ (a vector of d features).
- Each output: $y_i \in \mathcal{Y}$ (the target/label).

Goal: Find a function $f : \mathbb{R}^d \to \mathcal{Y}$ that maps inputs to outputs, such that f generalizes well to **unseen data**.

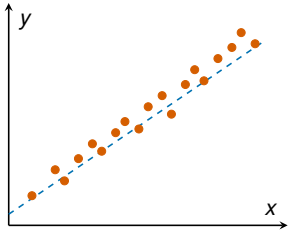
Two types of problems:

Regression: $y \in \mathbb{R}$ (continuous)
e.g., predict GDP growth, house prices.

Classification: $y \in \{0, 1, \ldots, K\}$ (discrete)
e.g., spam detection, recession indicator.

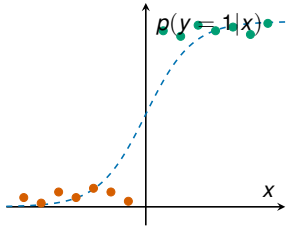
Regression vs. Classification

Regression: y is continuous



$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$$

Classification: y is binary/categorical



$$p(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

In both cases: we choose a **model**, define a **loss**, and **optimize**.

Connection to Econometrics

What you already know

OLS regression is a special case of supervised learning:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_d x_{id} + \varepsilon_i$$

- Model: $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ (linear hypothesis).
- Loss: $\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (y_i - f(\mathbf{x}_i))^2$ (MSE = OLS objective).
- Optimization: closed-form solution $\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.

What deep learning adds:

- Replace the **linear model** with a non-linear, multi-layer function.
- Replace the **closed-form solution** with iterative optimization (gradient descent).
- Replace **hand-crafted features** with learned representations.

The Hypothesis Set

Definition

A **hypothesis set** $\mathcal{H}$ is the space of candidate functions the learning algorithm can choose from to map inputs $\mathbf{x}$ to outputs y.

Examples:

Model	Hypothesis set $\mathcal{H}$	Complexity
Linear regression	$\{f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b : \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}\}$	Low
Polynomial	$\{f(\mathbf{x}) = \sum_{\lvert\alpha\rvert \le p} w_\alpha \mathbf{x}^\alpha\}$	Medium
Neural network	$\{f(\mathbf{x}) = \mathbf{W}^{(L)}\sigma(\cdots\sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})\cdots) + \mathbf{b}^{(L)}\}$	High

Deep learning = very rich hypothesis set + data-driven optimization.

Training Data vs. Test Data

Training set $\mathcal{D}_{\text{train}}$:

- Used to learn the model parameters.
- We observe both **x** and y.

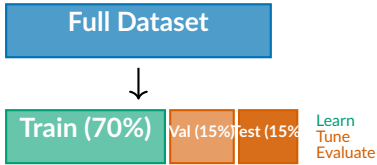
Test set $\mathcal{D}_{\text{test}}$:

- Used *only* to evaluate generalization.
- Never use test data to modify the model.

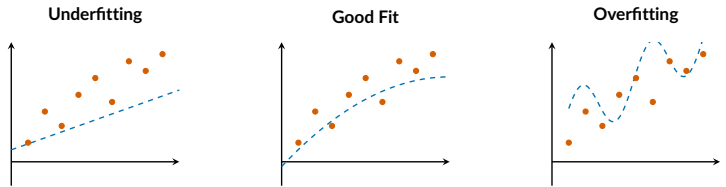
Validation set $\mathcal{D}_{\text{val}}$:

- Used to tune hyperparameters.
- Select model complexity.

Key Principle: The goal is generalization — performing well on data the model has *never seen*.



Overfitting vs. Underfitting



Overfitting: the model memorizes training data (including noise) and fails to generalize.
Prevention: regularization (L1, L2), early stopping, dropout, more data.

Deep networks have millions of parameters — overfitting is a central concern.

Linear Models: The Starting Point

Linear Regression

Model

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$$

where $\mathbf{w} \in \mathbb{R}^d$ is the **weight vector** and $b \in \mathbb{R}$ is the **bias**.

Loss function: Mean Squared Error (MSE)

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - f(\mathbf{x}_i)\right)^2 = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - \mathbf{w}^\top \mathbf{x}_i - b\right)^2$$

Goal: Find the parameters $(\mathbf{w}^*, b^*)$ that minimize the loss:

$$\mathbf{w}^*, b^* = \arg\min_{\mathbf{w}, b} \; \mathcal{L}(\mathbf{w}, b)$$

Closed-form solution (via $\nabla_{\mathbf{w}} \mathcal{L} = 0$): $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.

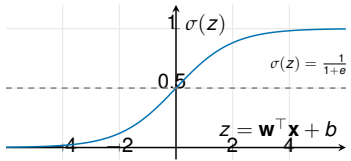
This is exactly OLS — the starting point for all of deep learning.

Logistic Regression for Classification

For **binary classification** ($y \in \{0, 1\}$), we need $f(\mathbf{x}) \in [0, 1]$.

Step 1: Apply the **sigmoid function** σ to the linear model:

$$p(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$



Properties: $\sigma(0) = 0.5$, $\lim_{z\to\infty} \sigma(z) = 1$, $\lim_{z\to-\infty} \sigma(z) = 0$, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Binary Cross-Entropy Loss

For classification, MSE is not ideal. Instead, we use the **binary cross-entropy** loss:

Binary Cross-Entropy (BCE)

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\Big]$$

where $\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$ is the predicted probability.

Intuition:

- If $y_i = 1$: loss $= -\log \hat{y}_i$ $\Rightarrow$ penalizes when $\hat{y}_i \approx 0$ (confident wrong).
- If $y_i = 0$: loss $= -\log(1-\hat{y}_i)$ $\Rightarrow$ penalizes when $\hat{y}_i \approx 1$ (confident wrong).

Derivation: comes from **maximum likelihood estimation** (MLE):

$$\max_{\mathbf{w},b} \prod_{i=1}^{N} \hat{y}_i^{y_i}(1-\hat{y}_i)^{1-y_i} \quad \Leftrightarrow \quad \min_{\mathbf{w},b} \mathcal{L}(\mathbf{w}, b)$$

Gradient Descent

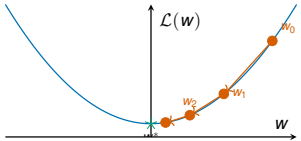
Unlike linear regression, most models **don't have closed-form solutions**. We use **iterative optimization**.

Gradient Descent Update Rule

Repeat until convergence:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}, b) \qquad b \leftarrow b - \alpha \frac{\partial \mathcal{L}}{\partial b}$$

where $\alpha > 0$ is the **learning rate**.



Intuition: move in the direction of steepest descent, with step size α.

Gradient Derivation: Linear Regression Example

Let $f(x) = wx + b$ with a single input ($d = 1$). Loss for one sample:

$$\ell = (y - wx - b)^2$$

Computing the gradients:

$$\frac{\partial \ell}{\partial w} = \frac{\partial}{\partial w}(y - wx - b)^2 = 2(y - wx - b) \cdot (-x) = -2x(y - wx - b)$$

$$\frac{\partial \ell}{\partial b} = \frac{\partial}{\partial b}(y - wx - b)^2 = 2(y - wx - b) \cdot (-1) = -2(y - wx - b)$$

For the full dataset (average over N samples):

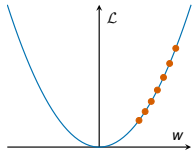
$$\frac{\partial \mathcal{L}}{\partial w} = -\frac{2}{N}\sum_{i=1}^{N} x_i(y_i - wx_i - b) \qquad \frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{N}\sum_{i=1}^{N}(y_i - wx_i - b)$$

Update:

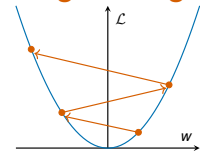
$$w \leftarrow w + \frac{2\alpha}{N}\sum_{i=1}^{N} x_i(y_i - wx_i - b) \qquad b \leftarrow b + \frac{2\alpha}{N}\sum_{i=1}^{N}(y_i - wx_i - b)$$

The Learning Rate α

Too small: slow convergence



Too large: divergence

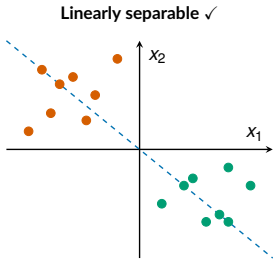


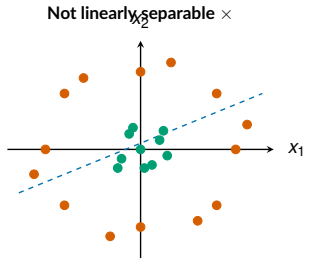
α **too small**	Converges, but very slowly
α **too large**	Overshoots, may diverge
α **just right**	Converges efficiently

Typical starting values: $\alpha \in \{0.001, 0.01, 0.1\}$. We'll learn better strategies in Lecture 3.

The Limitation of Linear Models

Linear models can only produce **linear decision boundaries**.





No single line works!

We need **non-linear** models. This is where neural networks come in.

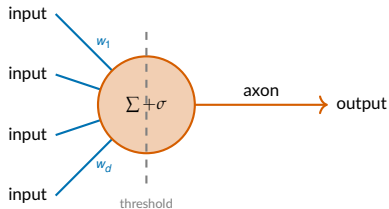
The Perceptron

Biological Inspiration

Biological neuron:

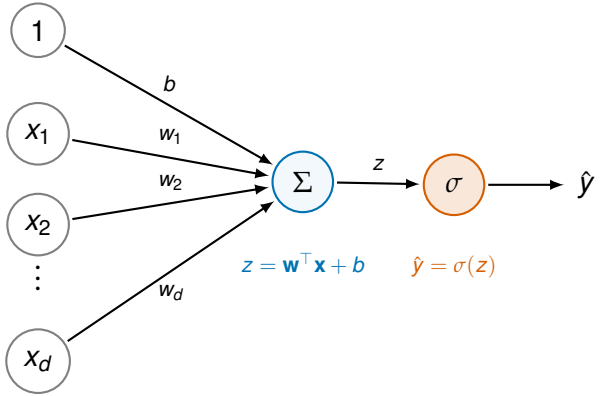
- Receives input signals from **dendrites**.
- Processes them in the **cell body**.
- If total signal exceeds a threshold, **fires** through the **axon**.
- Connections have different **strengths** (synaptic weights).

McCulloch & Pitts (1943) proposed the first mathematical model of a neuron.



The analogy is **loose**. Real neurons are far more complex. But the abstraction is powerful.

The Artificial Neuron (Perceptron)



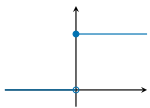
The Perceptron

$$\hat{y} = \sigma\left(\sum_{j=1}^{d} w_j x_j + b\right) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

Activation Functions

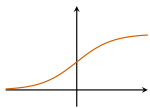
The activation function σ introduces **non-linearity** into the model.

Step (Heaviside)



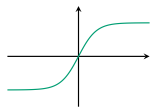
$$\sigma(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

Sigmoid



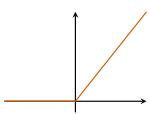
$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Tanh



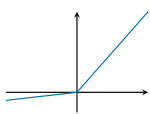
$$\sigma(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$$

ReLU (most popular)



$$\sigma(z) = \max(0, z)$$

Leaky ReLU



$$\sigma(z) = \max(0.01z, z)$$

Why Do We Need Non-linearity?

Critical Insight: Without activation functions, a multi-layer network collapses to a **single linear transformation**.

Proof: Consider two linear layers (no activation):

$$\mathbf{h}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\hat{\mathbf{y}} = \mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)} = \mathbf{W}^{(2)}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)}$$

$$= \underbrace{\mathbf{W}^{(2)}\mathbf{W}^{(1)}}_{\mathbf{W}'}\mathbf{x} + \underbrace{\mathbf{W}^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)}}_{\mathbf{b}'} = \mathbf{W}'\mathbf{x} + \mathbf{b}'$$

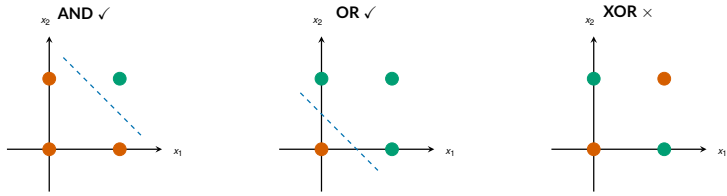
This is just **another linear model**! The depth is useless.

Non-linear activations are what make depth meaningful.

With activations: $\hat{\mathbf{y}} = \mathbf{W}^{(2)}\sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)}$ $\Rightarrow$ genuinely non-linear.

The XOR Problem: Why One Layer Is Not Enough

Boolean operations with a single perceptron:

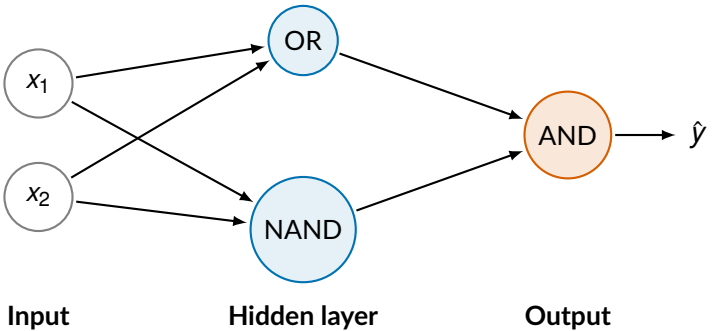


No single line!

- AND and OR are **linearly separable** — a single perceptron can learn them.
- XOR is **not linearly separable** — requires composing multiple linear boundaries.
- Shown by **Minsky & Papert (1969)** and caused the first “AI winter”.

Solving XOR with Two Layers

Key insight: XOR $=$ AND(OR(x_1, x_2), NAND(x_1, x_2))



Adding one hidden layer solves the problem! This is the power of depth.

XOR: Truth Table Verification

The hidden layer computes OR and NAND; the output layer applies AND.

x_1	x_2	OR(x_1, x_2)	NAND(x_1, x_2)	AND(OR, NAND) = **XOR**
0	0	0	1	**0**
0	1	1	1	**1**
1	0	1	1	**1**
1	1	1	0	**0**

- The hidden layer **transforms** the input into a new representation.
- In the new representation, the two classes **are** linearly separable.
- This is the essence of deep learning: learn useful representations, then classify.

XOR: The Explicit Computation

Using step activation $\sigma(z) = \mathbb{1}[z > 0]$:

Hidden layer:

$$h_1 = \sigma(x_1 + x_2 - 0.5) \qquad \text{(OR: fires if } x_1 + x_2 > 0.5\text{)}$$
$$h_2 = \sigma(-x_1 - x_2 + 1.5) \qquad \text{(NAND: fires if } x_1 + x_2 < 1.5\text{)}$$

Output layer:

$$\hat{y} = \sigma(h_1 + h_2 - 1.5) \qquad \text{(AND: fires if } h_1 + h_2 > 1.5\text{)}$$

In matrix form:

$$\mathbf{W}^{(1)} = \begin{pmatrix} 1 & 1 \\ -1 & -1 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} -0.5 \\ 1.5 \end{pmatrix}, \quad \mathbf{W}^{(2)} = \begin{pmatrix} 1 & 1 \end{pmatrix}, \quad b^{(2)} = -1.5$$

Verify for $(x_1, x_2) = (1, 0)$:

$$\mathbf{h} = \sigma\left(\begin{pmatrix} 1 & 1 \\ -1 & -1 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} -0.5 \\ 1.5 \end{pmatrix}\right) = \sigma\begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$\hat{y} = \sigma(1 + 1 - 1.5) = \sigma(0.5) = 1 \quad \checkmark$$

From Perceptrons to Deep Neural Networks

From One Neuron to a Layer

One neuron: computes a single output feature

$$h_j = \sigma\left(\sum_{k=1}^{d} w_{jk}\, x_k + b_j\right) = \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j)$$

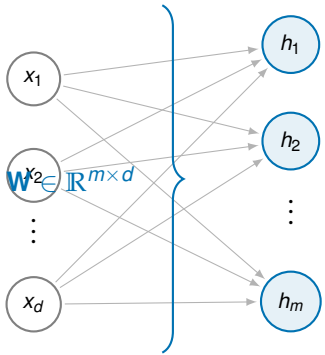
A layer of m neurons: computes m output features simultaneously

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) = \begin{pmatrix} \sigma(\mathbf{w}_1^\top \mathbf{x} + b_1) \\ \sigma(\mathbf{w}_2^\top \mathbf{x} + b_2) \\ \vdots \\ \sigma(\mathbf{w}_m^\top \mathbf{x} + b_m) \end{pmatrix} \in \mathbb{R}^m$$

where $\mathbf{W} \in \mathbb{R}^{m \times d}$ and $\mathbf{b} \in \mathbb{R}^m$.

Each neuron in the layer learns a **different linear boundary**; the activation function makes each one non-linear.

A Fully Connected Layer



$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

$$h_j = \sigma\left(\sum_{k=1}^{d} W_{jk}\, x_k + b_j\right)$$

Every input is connected to every neuron: this is called a **fully connected** (or **dense**) layer. Parameters: $m \times d$ weights + m biases $= m(d+1)$ total.

Stacking Layers: The Deep Neural Network

A feedforward neural network is a composition of layers:

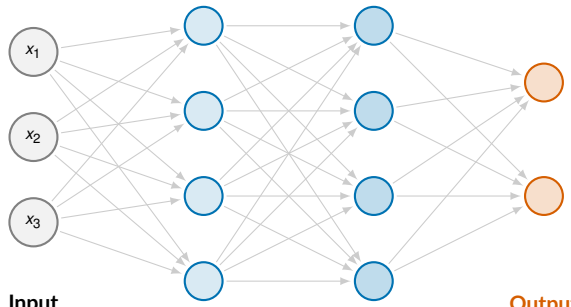
Feedforward Neural Network with L layers

$$
\begin{aligned}
\mathbf{h}^{(0)} &= \mathbf{x} && \text{(input)} \\
\mathbf{h}^{(1)} &= \sigma(\mathbf{W}^{(1)}\mathbf{h}^{(0)} + \mathbf{b}^{(1)}) && \text{(hidden layer 1)} \\
\mathbf{h}^{(2)} &= \sigma(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}) && \text{(hidden layer 2)} \\
&\vdots \\
\mathbf{h}^{(L-1)} &= \sigma(\mathbf{W}^{(L-1)}\mathbf{h}^{(L-2)} + \mathbf{b}^{(L-1)}) && \text{(hidden layer } L-1\text{)} \\
\hat{\mathbf{y}} &= \mathbf{W}^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)} && \text{(output layer)}
\end{aligned}
$$

Terminology:

- **Depth:** number of layers L ("deep" $= L \geq 2$ hidden layers).

Deep Neural Network: Diagram



Parameters:
$\mathbf{W}^{(1)} : 4 \times 3 = 12$
$\mathbf{b}^{(1)} : 4$
$\mathbf{W}^{(2)} : 4 \times 4 = 16$
$\mathbf{b}^{(2)} : 4$
$\mathbf{W}^{(3)} : 2 \times 4 = 8$
$\mathbf{b}^{(3)} : 2$
Total: 46

Modern networks: GPT-4 has $\sim$1.8 **trillion** parameters. ResNet-50 has $\sim$25 million.

Forward Pass: A Numerical Example

Network: 2 inputs $\to$ 2 hidden (ReLU) $\to$ 1 output (linear)

Given: $\mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $\mathbf{W}^{(1)} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.8 \end{pmatrix}$, $\mathbf{b}^{(1)} = \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix}$, $\mathbf{W}^{(2)} = \begin{pmatrix} 0.6 & -0.4 \end{pmatrix}$, $b^{(2)} = 0.2$

Step 1: Compute hidden layer pre-activation

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.8 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix} = \begin{pmatrix} -0.1 \\ 1.7 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix} = \begin{pmatrix} 0.0 \\ 1.6 \end{pmatrix}$$

Step 2: Apply ReLU activation

$$\mathbf{h}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)}) = \begin{pmatrix} \max(0, 0.0) \\ \max(0, 1.6) \end{pmatrix} = \begin{pmatrix} 0.0 \\ 1.6 \end{pmatrix}$$

Step 3: Compute output

$$\hat{y} = \mathbf{W}^{(2)}\mathbf{h}^{(1)} + b^{(2)} = \begin{pmatrix} 0.6 & -0.4 \end{pmatrix} \begin{pmatrix} 0.0 \\ 1.6 \end{pmatrix} + 0.2 = -0.64 + 0.2 = -0.44$$

Dimensions Cheat Sheet

For a network with layers of widths $d = m^{(0)}, m^{(1)}, m^{(2)}, \ldots, m^{(L)}$:

Layer l	$\mathbf{W}^{(l)}$	$\mathbf{b}^{(l)}$	$\mathbf{h}^{(l)}$
Input ($l = 0$)	—	—	$\mathbb{R}^{m^{(0)}}$
Hidden l	$\mathbb{R}^{m^{(l)} \times m^{(l-1)}}$	$\mathbb{R}^{m^{(l)}}$	$\mathbb{R}^{m^{(l)}}$
Output ($l = L$)	$\mathbb{R}^{m^{(L)} \times m^{(L-1)}}$	$\mathbb{R}^{m^{(L)}}$	$\mathbb{R}^{m^{(L)}}$

Total parameters:

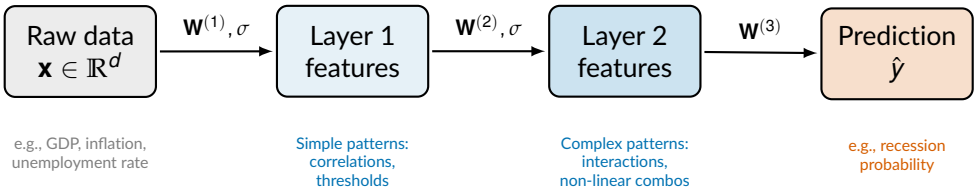
$$\sum_{l=1}^{L} \left(m^{(l)} \times m^{(l-1)} + m^{(l)} \right) = \sum_{l=1}^{L} m^{(l)} \left(m^{(l-1)} + 1 \right)$$

Example: Input 784 $\rightarrow$ Hidden 256 $\rightarrow$ Hidden 128 $\rightarrow$ Output 10

$$(256 \times 785) + (128 \times 257) + (10 \times 129) = 200{,}960 + 32{,}896 + 1{,}290 = 235{,}146$$

What Does a Neural Network Learn?

Key idea: each layer learns a new representation of the data.



For economists: think of it as **automatic feature engineering**:

- Instead of manually creating interaction terms ($x_1 \cdot x_2$, x_1^2, etc.).
- The network **learns** which transformations of the raw features are useful.
- Particularly powerful when d is large (hundreds of variables).

Introduction to PyTorch & Google Colab

Why PyTorch?

PyTorch is the most widely used deep learning framework in research.

Key features:

- **Dynamic computation graphs** (define-by-run).
- **Automatic differentiation** (autograd).
- GPU acceleration (CUDA).
- Pythonic, intuitive API.
- Huge ecosystem (torchvision, torchtext, HuggingFace).

Alternatives:

- TensorFlow / Keras (Google).
- JAX (Google Research).
- Flux.jl (Julia).

We use PyTorch because:

- Standard in academia.
- Best for learning DL concepts.
- HuggingFace integration (for LLMs).

PyTorch Tensors

Tensors are the fundamental data structure — like NumPy arrays but with GPU support.

```python
import torch

# Creating tensors
x = torch.tensor([1.0, 2.0, 3.0])          # 1D tensor (vector)
W = torch.randn(3, 2)                       # 2D tensor (matrix), random
b = torch.zeros(3)                          # 1D tensor of zeros

# Operations
z = W @ x + b                  # Matrix-vector multiply + bias (@ = matmul)
print(z.shape)                 # torch.Size([3])

# Move to GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
x_gpu = x.to(device)
W_gpu = W.to(device)
z_gpu = W_gpu @ x_gpu          # Computation happens on GPU!
```

Key difference from NumPy: tensors track computation for automatic differentiation.

Automatic Differentiation (Autograd)

PyTorch can automatically compute gradients — the foundation of training neural networks.

```python
import torch

# Create tensors with gradient tracking
w = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
x = torch.tensor(3.0)
y_true = torch.tensor(7.0)      # target

# Forward pass: compute prediction and loss
y_pred = w * x + b              # y_pred = 2*3 + 1 = 7
loss = (y_true - y_pred) ** 2   # MSE for one sample = 0

# Backward pass: compute gradients
loss.backward()

# Gradients are now available!
print(w.grad)     # d(loss)/dw = 2*(y_true - y_pred)*(-x) = 0
print(b.grad)     # d(loss)/db = 2*(y_true - y_pred)*(-1) = 0
```

Key Insight: `loss.backward()` computes $\frac{\partial \mathcal{L}}{\partial w}$ and $\frac{\partial \mathcal{L}}{\partial b}$ **automatically** using the chain rule. We'll derive backpropagation in Lecture 2.

Implementing a Single Neuron in PyTorch

```python
import torch
import torch.nn as nn

# A single neuron: 2 inputs -> 1 output with sigmoid activation
class SingleNeuron(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(2, 1)     # 2 inputs, 1 output
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        z = self.linear(x)                # z = w^T x + b
        return self.sigmoid(z)            # sigma(z)

# Create the neuron and a sample input
neuron = SingleNeuron()
x = torch.tensor([[1.0, 2.0]])          # batch of 1, 2 features
y_pred = neuron(x)
print(f"Prediction: {y_pred.item():.4f}")

# Check parameters
for name, param in neuron.named_parameters():
    print(f"{name}: {param.data}")
```

The Training Loop (Preview)

```python
# The basic training loop structure
model = SingleNeuron()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)  # SGD
criterion = nn.BCELoss()                                   # Binary CE

for epoch in range(100):
    # Forward pass
    y_pred = model(x_train)
    loss = criterion(y_pred, y_train)

    # Backward pass
    optimizer.zero_grad()     # Reset gradients to zero
    loss.backward()           # Compute gradients (backpropagation)

    # Update parameters
    optimizer.step()          # w <- w - alpha * grad

    if epoch % 10 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item():.4f}")
```

Every DL training follows this pattern: forward → loss → backward → update.
We'll understand *exactly why* in Lecture 2 (backpropagation).

Google Colab Setup

Google Colab provides free access to GPUs in the browser.

Getting started:

1. Go to `colab.research.google.com`.
2. Create a new notebook.
3. Runtime $\rightarrow$ Change runtime type $\rightarrow$ **GPU**.
4. PyTorch is pre-installed!

Recommended subscriptions:

- **Colab Pro** ($10/mo): better GPUs, longer runtime.
- **Colab Pro+** ($50/mo): even more resources.
- Free tier works for Lectures 1–5.

Verify GPU access:

```
1 import torch
2 print(torch.cuda.is_available())
3 # True
4 print(torch.cuda.get_device_name(0)
      )
5 # Tesla T4
```

Also recommended:

- **Claude Code** ($20/mo) for coding assistance.
- Use LLMs as a learning tool, not a replacement.

Summary & Next Steps

Lecture 1 Summary

1. **Supervised learning:** given data $\{(\mathbf{x}_i, y_i)\}$, find f that generalizes.
2. **Linear models** (regression/logistic) are the starting point:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b \qquad \text{or} \qquad p(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

3. **Gradient descent** iteratively minimizes the loss:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} \mathcal{L}$$

4. **The perceptron** = linear model + activation function. Learns AND, OR but not XOR.
5. **Deep neural networks** stack layers of perceptrons:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$$

6. **Non-linear activations** (ReLU, sigmoid) are essential — without them, depth is useless.
7. **PyTorch** provides tensors + autograd for implementation.

Next Lecture: Backpropagation & Gradient Descent

Lecture 2 (Friday, April 17):

- How does the network actually **learn**?
- The **chain rule** and backpropagation algorithm.
- Full derivation of gradients for multi-layer networks.
- Loss functions in detail: MSE, cross-entropy, softmax.
- Gradient descent variants.

Recommended reading:

- Goodfellow et al., *Deep Learning*, Chapters 6.1–6.3.
- Zhang et al., *Dive into Deep Learning*, Chapters 3–4 (`https://d2l.ai`).

Homework 1 will be released after Lecture 3 (April 20).

Questions?

References

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into Deep Learning*. Cambridge University Press.
- Rosenblatt, F. (1958). The Perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386.
- Minsky, M., & Papert, S. (1969). *Perceptrons*. MIT Press.
- Jean, N. et al. (2016). Combining Satellite Imagery and Machine Learning to Predict Poverty. *Science*, 353(6301), 790–794.
- Gentzkow, M., Kelly, B., & Taddy, M. (2019). Text as Data. *Journal of Economic Literature*, 57(3), 535–574.
- Athey, S., & Imbens, G. W. (2019). Machine Learning Methods That Economists Should Know About. *Annual Review of Economics*, 11, 685–725.